RELATÓRIO DE AUDITORIA DE SEGURANÇA

Sistema Leitores de Papel — IFBA Ilhéus

Data da Avaliação:	01 de Outubro de 2026
Organização:	Instituto Federal de Educação, Ciência e Tecnologia da Bahia (IFBA - Campus Ilhéus)
Alvo do Escopo:	Repositório Completo (Diretórios public/, admin/, user/, actions/, config/, includes/)
Classificação:	**CONFIDENCIAL / AUDITORIA DE CÓDIGO FONTE (SAST)**
Status da Análise:	Concluída (100% de cobertura dos endpoints e handlers)

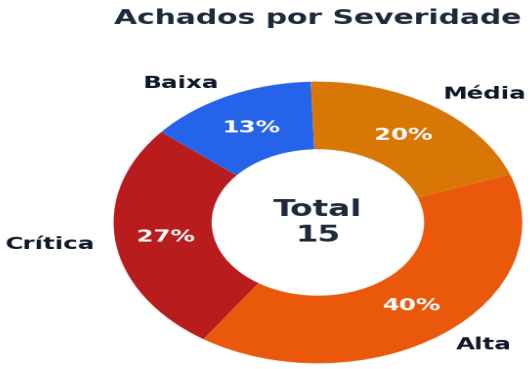
1. Detecção da Stack & Nota Metodológica

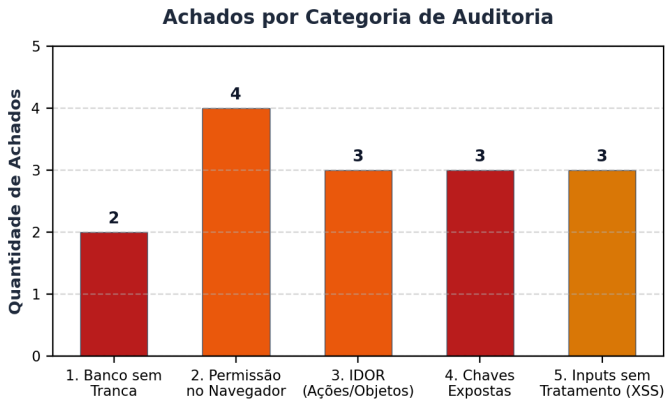
Stack Tecnológica Detectada:
• **Linguagem & Paradigma:** PHP Procedural (compatível com PHP 5.6 e PHP 8.2).
• **Framework:** Nenhum (arquitetura Vanilla PHP com roteamento baseado em arquivos físicos).
• **Banco de Dados / ORM:** MySQL Server acessado via extensão nativa mysqli (sem ORM / sem Query Builder).
• **Autenticação & Sessão:** Sessões nativas do PHP ($_SESSION), hashes de senha com password_hash() BCRYPT e password_verify().
• **Frontend:** HTML5, CSS customizado + Bootstrap 4, JavaScript Vanilla e FontAwesome via CDN.
• **Ambiente de Execução:** Servidor Web Apache rodando sob MAMP PRO localmente e infraestrutura IFBA em produção.

Adaptação Metodológica das Categorias Auditadas:
1. **Banco sem Tranca:** Em aplicações sem RLS (como Supabase), a segurança reside nas cláusulas WHERE das consultas SQL que devem obrigatoriamente restringir o escopo ao $_SESSION['user_id']. Verificou-se onde leituras/mutações operam sem essa barreira.
2. **Permissão Definida no Navegador:** Confrontou-se o modelo de menus no frontend (onde links administrativos ficam ocultos para alunos) com as validações de backend nos scripts de admin/ e actions/, identificando endpoints privilegiados que aceitam requisições sem validar privilégio administrativo.
3. **IDOR (Insecure Direct Object Reference):** Análise manual e sistemática de todos os parâmetros livro_id, id e user_id em endpoints de leitura, escrita e exclusão.
4. **Chaves Expostas:** Varredura em busca de tokens SMTP, senhas de banco e credenciais embutidas diretamente no código.
5. **Inputs sem Tratamento (XSS & Injeções):** Identificação de saídas de $_GET, $_POST e dados do banco renderizados sem htmlspecialchars(), além de concatenação de strings em SQL.

2. Resumo Executivo

A auditoria de segurança estática (SAST) realizada no sistema **Leitores de Papel** identificou **15 vulnerabilidades ativas** distribuídas em todas as 5 categorias avaliadas. O sistema apresenta pontos arquiteturais positivos no armazenamento criptográfico de senhas, porém possui falhas críticas de **controle de acesso quebrado (Broken Access Control)**, permitindo que usuários comuns ou até mesmo atacantes desautenticados manipulem o acervo, excluam livros, acessem dados sensíveis de terceiros e utilizem credenciais corporativas vazadas.





CRÍTICA: 4	ALTA: 6	MÉDIA: 3	BAIXA: 2	PONTOS FORTES: 5

3. Análise de Postura de Segurança

3.1. Pontos Fortes Verificados (Evidências Positivas)

• **Armazenamento Seguro de Credenciais:** Em public/cadastro.php:41 e actions/login.php:56, o sistema utiliza password_hash(..., PASSWORD_DEFAULT) gerando hashes BCRYPT robustos com salt automático, e autentica via password_verify(). Não há uso de algoritmos defasados (MD5/SHA1).
• **Autenticação com Prepared Statements:** Em actions/login.php:23-35, as consultas de login para usuários e administradores utilizam $conn->prepare() com binding tipado (bind_param('s', $username)), impedindo SQL Injection na tela de login.
• **Inserção de Comentários Parametrizada:** Em actions/salvar_comentario_action.php:18-20, a query de inclusão de avaliações utiliza prepared statements com amarração estrita de tipos (iiss).
• **Escopo Seguro de Atualização de Perfil de Aluno:** Em user/atualiza_dados.php:38-50, a alteração de dados pessoais restringe-se a WHERE username = '$_SESSION[username]', não permitindo edição de terceiros por injeção de ID.
• **Escape no Cabeçalho de Usuário:** Em includes/header.php:42, 85, o nome do usuário exibido na barra superior é protegido com htmlspecialchars(), prevenindo XSS na saudação.

3.2. Riscos Centrais e Pontos Fracos Críticos

• **Controle de Acesso em Nível de Função Inexistente:** Scripts administrativos críticos (como admin/excluir_livro.php, actions/salvar_livro.php e admin/informacoes_usuarios.php) não verificam a existência de sessão administrativa. No painel principal (admin/index.php:4), o check lógico !isset(admin) && !isset(user) concede acesso a alunos.
• **Vazamento Direto de Credenciais em Código (Hardcoded):** Chaves SMTP do provedor Brevo (usuário e senha do Gmail corporativo) estão commitadas em texto plano em admin/notificacao.php:50-52, viabilizando o sequestro do relay de e-mail.
• **Manipulação Indireta de Estoque e IDOR:** Em actions/devolve_livro_action.php:37-65, a devolução incrementa o acervo de qualquer livro antes de verificar posse, permitindo adulteração maliciosa do inventário da biblioteca.
• **Superfície Ampla de XSS Refletido:** Parâmetros de URL (como error e success_message) em public/index.php e parâmetros de pesquisa em admin/lista_livros.php são concatenados sem sanitização no DOM.

4. Tabela de Achados Detalhados por Categoria

Abaixo constam todos os 15 achados verificados diretamente no código-fonte, categorizados e classificados por criticidade.

Sev.	Cat.	Arquivo : Linha	Descrição da Vulnerabilidade & Impacto
[CRÍTICA]	**1. Banco sem Tranca**	actions/devolve_livro_action.php:37-68	**Manipulação de Estoque sem Validação de Posse:** O script incrementa o estoque do livro via UPDATE livros SET quantidade+1 antes de verificar se o usuário autenticado realmente possui o empréstimo ativo. Atacante autenticado pode inflar arbitrariamente o estoque de qualquer livro.
[ALTA]	**1. Banco sem Tranca**	actions/atualizar_emprestimos.php:19-65	**Sincronização de Histórico Global Irrestrita:** Handler executa sincronização indiscriminada de toda a tabela livros_emprestados sem isolar pelo user_id do usuário que disparou a ação, misturando registros de múltiplos usuários.
[CRÍTICA]	**2. Permissão no Navegador**	admin/excluir_livro.php:9-20	**Exclusão Pública de Livros sem Autenticação:** O endpoint aceita requisições POST para deleção de livros diretamente do banco (DELETE FROM livros WHERE id = $livro_id) sem validar sessão nem privilégio de administrador.
[CRÍTICA]	**2. Permissão no Navegador**	admin/index.php:4-7	**Bypass de Autenticação no Painel Administrador:** Condição de acesso usa operador &&: !isset($_SESSION['admin_username']) && !isset($_SESSION['username']). Qualquer aluno autenticado com $_SESSION['username'] obtém acesso irrestrito ao painel administrativo.
[ALTA]	**2. Permissão no Navegador**	admin/lista_usuarios.php:13-19	**Vazamento de PII de Usuários para Alunos:** Página de gerenciamento administrativo valida apenas !isset($_SESSION['user_id']). Alunos podem acessar a rota e extrair nome, matrícula, e-mail, telefone, sexo e data de nascimento de toda a base de usuários.
[ALTA]	**2. Permissão no Navegador**	actions/salvar_livro.php:13-42	**Criação de Livros sem Autenticação:** Ação de cadastro de livros no acervo processa dados de formulário e executa INSERT sem validar se o usuário é administrador ou se sequer está autenticado.

[ALTA]	**2. Permissão no Navegador**	admin/editar_livro.php:13-55	**Edição e Exclusão de Livros por Usuários Comuns:** O script administrativo valida apenas se $_SESSION['user_id'] existe, permitindo que leitores comuns alterem metadados de qualquer obra ou excluam livros desativando chaves estrangeiras.
[ALTA]	**2. Permissão no Navegador**	admin/informacoes_usuarios.php:98-120	**Relatório de Empréstimos Acessível Publicamente:** O script não possui nenhum bloqueio de autenticação. Qualquer visitante desautenticado na internet pode pesquisar e listar todos os empréstimos registrados com nomes dos estudantes.
[ALTA]	**3. IDOR**	actions/atualizar_livro.php:23-50	**IDOR na Atualização de Obras:** O parâmetro POST['livro_id'] é utilizado diretamente na query UPDATE livros ... WHERE id='$livro_id' sem checar privilégios administrativos no backend.
[MÉDIA]	**3. IDOR**	actions/salvar_comentario_action.php:10-25	**Inserção de Comentário com Identidade Nula/Falsificada:** Script faz cast (int)$_SESSION['user_id'] sem validar se a sessão existe. Permite inserção de comentários vinculados a user_id = 0 sem validação se o usuário já leu ou pegou a obra.
[MÉDIA]	**3. IDOR**	admin/devolve_livro.php:25-45	**IDOR na Ação de Devolução Administrativa:** Handler aceita livro_id via POST e remove registro sem validar se o empréstimo pertencia à transação esperada.
[CRÍTICA]	**4. Chaves Expostas**	admin/notificacao.php:50-52	**Credenciais SMTP Brevo em Texto Claro:** Host smtp-relay.brevo.com, conta nzgamebr@gmail.com e senha 0LQ98cwOraSE7RX2 commitadas no repositório. Permite roubo de serviço SMTP e envio de phishing.
[ALTA]	**4. Chaves Expostas**	public/recuperar_senha.php:72-73	**Credenciais de Sandbox Mailtrap Expostas:** Usuário 89f6fb8d8f567c e senha 2174786544ed23 gravadas no código em páginas de recuperação de senha (também em admin/recuperar_senha.php:67-68).
[MÉDIA]	**4. Chaves Expostas**	public/index.php:178-183	**Exibição Pública de Credencial Default na UI:** A tela de login exibe textualmente Usuário Admin: admin / Senha: admin para qualquer visitante que acesse o portal.
[MÉDIA]	**5. Inputs sem Tratamento**	public/index.php:104, 112	**Reflected XSS via Parâmetros de URL:** Os parâmetros $_GET['success_message'] e $_GET['error'] são impressos diretamente no corpo da página com echo sem htmlspecialchars().
[BAIXA]	**5. Inputs sem Tratamento**	admin/lista_livros.php:252, 261, 270	**Reflected XSS em Atributos de Filtro:** Parâmetros de busca titulo, autor e ano são injetados diretamente em atributos HTML value="...".
[BAIXA]	**5. Inputs sem Tratamento**	public/cadastro.php:95	**Reflected XSS em PHP_SELF:** Concatenação de $_SERVER['PHP_SELF'] no atributo action do formulário de cadastro sem sanitização.

5. Recomendações Priorizadas de Remediação

As correções devem seguir a matriz de risco abaixo para mitigar de imediato as brechas mais severas:

Prioridade	Ação Recomendada	Impacto / Mitigação
P1 (Crítica)	**Revogação Imediata de Credenciais e Isolamento de Segredos:** 1. Revogar e alterar a senha da conta Brevo (Sendinblue) exposta em admin/notificacao.php. 2. Criar arquivo config/secrets.php (adicionado ao .gitignore) ou usar variáveis de ambiente (getenv) para SMTP e banco de dados. 3. Remover o aviso com credenciais padrão em public/index.php e obrigar troca de senha no primeiro login.	Impede o sequestro da infraestrutura de e-mail e invasão imediata do portal via credenciais default conhecidas.
P2 (Alta)	**Implementação de Middleware/Gate Central de Autorização:** 1. Criar função require_admin() que valida estritamente isset($_SESSION['admin_username']) e aborta com HTTP 403 caso contrário. 2. Aplicar essa proteção no topo de **todos** os arquivos em admin/ e nos handlers de escrita correspondentes em actions/. 3. Corrigir a condição em admin/index.php para impedir acesso de estudantes.	Elimina a escalação horizontal e vertical de privilégios de alunos para administrador.
P3 (Alta)	**Correção de Integridade de Empréstimos e IDOR:** 1. Em actions/devolve_livro_action.php, validar atomicamente com transação SQL se o registro existe em livros_emprestados para aquele user_id antes de decrementar/incrementar estoque. 2. Usar transações ($conn->begin_transaction()) para garantir consistência.	Elimina a manipulação indevida de acervo e adulteração do inventário da biblioteca.
P4 (Média)	**Sanitização Global de Saídas (Prevenção de XSS):** 1. Envolver todas as variáveis refletidas em htmlspecialchars($var, ENT_QUOTES, 'UTF-8'). 2. Corrigir mensagens de erro e sucesso em public/index.php e campos de filtro em admin/lista_livros.php.	Impede a injeção e execução de scripts maliciosos na sessão dos usuários.

6. Issues Prontas para o GitHub

Os blocos abaixo contêm o texto completo em Markdown de cada issue acionável para inclusão no rastreador de tarefas (GitHub Issues). Cada issue agrupa achados relacionados com checklist e critérios de aceite.

```
--- ISSUE 1 ---
### [Segurança] Revogação e isolamento de credenciais SMTP e dados de autenticação expostos em código

**Labels recomendadas:** `security`, `severity:critical`, `credentials`

#### Descrição do Problema
Foram identificadas credenciais em texto claro embutidas diretamente no código-fonte do repositório,
incluindo conta SMTP do serviço Brevo (Sendinblue) com e-mail e senha corporativos, credenciais de
Mailtrap e exibição de credencial default de administrador na tela pública de login.

#### Evidências
- `admin/notificacao.php:50-52`:
```php
$mail->Username = 'nzgamebr@gmail.com';
$mail->Password = '0LQ98cwOraSE7RX2';
```
- `public/recuperar_senha.php:72-73` e `admin/recuperar_senha.php:67-68`:
```php
$mail->Username = '89f6fb8d8f567c';
$mail->Password = '2174786544ed23';
```
- `public/index.php:178-183`:
```html
<div class="alert alert-warning text-center" role="alert">
Usuário Admin: admin<br>Senha: admin
</div>
```

Impacto
- Uso malicioso do servidor SMTP Brevo para campanhas de spam/phishing em nome da instituição.
- Acesso imediato à área administrativa por qualquer visitante através da credencial padrão exposta.

Sugestão de Correção
1. Acessar o painel da Brevo e revogar imediatamente a chave SMTP exposta.
2. Criar um arquivo de configuração fora do controle de versão (ex: `config/env.php` no `.gitignore`) ou
ler de variáveis de ambiente do servidor (`getenv()`).
3. Remover a mensagem de credenciais padrão do `public/index.php`.

Critérios de Aceite
- [ ] Credenciais da Brevo revogadas e substituídas por credenciais seguras.
- [ ] Chaves de acesso lidas via variáveis de ambiente ou arquivo local protegido.
- [ ] Nenhuma senha ou chave presente no código-fonte rastreado pelo Git.
- [ ] Card de credencial default removido da tela de login.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
### [Segurança] Controle de Acesso Quebrado: Falta de validação de privilégio administrativo em endpoints
de admin e actions

**Labels recomendadas:** `security`, `severity:critical`, `access-control`, `rbac`

#### Descrição do Problema
O sistema esconde links administrativos no menu do leitor, mas a camada de backend não valida se a
requisição provém de um administrador. Em vários endpoints em `admin/` e `actions/`, o código valida
apenas `isset($_SESSION['user_id'])` (que qualquer aluno logado possui), ou não possui autenticação
alguma, ou contém erro de lógica condicional.

#### Evidências
- `admin/index.php:4`:
```php
if (!isset($_SESSION['admin_username']) && !isset($_SESSION['username'])) { ... }
```
```

```
Permite acesso ao painel de administração se `$_SESSION['username']` estiver definido (qualquer aluno
logado).

- `admin/excluir_livro.php:9-20`:
Processa `POST['livro_id']` e executa `DELETE FROM livros WHERE id = $livro_id` sem qualquer checagem de
sessão.

- `actions/salvar_livro.php:13-39`:
Permite que requisições POST criem livros no banco sem qualquer verificação de sessão ou papel.

- `admin/lista_usuarios.php:13-19` e `admin/editar_livro.php:13-19`:
Verificam apenas `!isset($_SESSION['user_id'])`, expondo dados confidenciais de alunos e edição de
acervo a qualquer leitor.

- `admin/informacoes_usuarios.php:98-120`:
Sem validação de sessão; relatório aberto publicamente.

#### Impacto
Alunos comuns e atacantes desautenticados podem ler dados de todos os usuários, cadastrar, modificar e
excluir livros do acervo, além de acessar painéis operacionais restritos.

#### Sugestão de Correção
1. Criar função auxiliar de autorização estrita (ex: `includes/auth.php`):
```php
function verificarAdmin() {
if (session_status() == PHP_SESSION_NONE) session_start();
if (empty($_SESSION['admin_username']) && empty($_SESSION['is_admin'])) {
http_response_code(403);
header("Location: ../public/index.php?error=" . urlencode("Acesso restrito a administradores."));
exit;
}
}
```
2. Invocar `verificarAdmin()` no início de todos os arquivos de `admin/` e nas ações de escrita de
catálogo em `actions/`.

Critérios de Aceite
- [ ] Acesso a qualquer URL em `admin/` redireciona para login caso o usuário seja leitor comum ou
anônimo.
- [ ] `actions/salvar_livro.php`, `actions/atualizar_livro.php` e `admin/excluir_livro.php` rejeitam
requisições sem sessão de administrador ativa.
- [ ] Relatórios de empréstimos e listagem de usuários inacessíveis para quem não for administrador.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
### [Segurança] IDOR e Adulteração de Estoque em Devolução de Obras

**Labels recomendadas:** `security`, `severity:high`, `idor`, `data-integrity`

#### Descrição do Problema
O endpoint de devolução de livros incrementa o estoque no banco de dados antes de validar se o usuário
solicitante realmente possui o exemplar emprestado. Um usuário malicioso pode enviar sucessivas
requisições POST com qualquer `livro_id` e inflar artificialmente o saldo de exemplares disponíveis na
biblioteca.

#### Evidências
- `actions/devolve_livro_action.php:37-68`:
```php
$query = "SELECT quantidade FROM livros WHERE id = '$livro_id'";
...
$update_query = "UPDATE livros SET quantidade = $nova_quantidade WHERE id = '$livro_id'";
$update_result = $conn->query($update_query);
if ($update_result) {
$delete_query = "DELETE FROM livros_emprestados WHERE livro_id = '$livro_id' AND user_id = '$user_id'";
$delete_result = $conn->query($delete_query);
```

Impacto
```

Corrupção do estoque do acervo, perda de controle de disponibilidade e dessincronização física/digital dos livros da instituição.

Sugestão de Correção
Validar a existência do empréstimo para o usuário antes de alterar a tabela de livros, utilizando transação atômica:
```php
$conn->begin_transaction();
$check = $conn->prepare("SELECT id FROM livros_emprestados WHERE livro_id = ? AND user_id = ? LIMIT 1");
$check->bind_param("ii", $livro_id, $user_id);
$check->execute();
if ($check->get_result()->num_rows === 1) {
$del = $conn->prepare("DELETE FROM livros_emprestados WHERE livro_id = ? AND user_id = ? LIMIT 1");
$del->bind_param("ii", $livro_id, $user_id);
$del->execute();
$upd = $conn->prepare("UPDATE livros SET quantidade = quantidade + 1 WHERE id = ?");
$upd->bind_param("i", $livro_id);
$upd->execute();
$conn->commit();
} else {
$conn->rollback();
}
```

Critérios de Aceite
- [ ] Devolução só altera quantidade do livro se o usuário comprovar posse ativa na tabela `livros_emprestados`.
- [ ] Operação executada sob transação ACID para evitar concorrência.
- [ ] Tentativas de devolver livros não emprestados retornam mensagem de erro sem alterar o estoque.
--- FIM ISSUE 3 ---

--- ISSUE 4 ---
[Segurança] Correção de Reflected Cross-Site Scripting (XSS) em Mensagens e Filtros

Labels recomendadas: `security`, `severity:medium`, `xss`

Descrição do Problema
Valores recebidos via `$_GET` e `$_SERVER['PHP_SELF']` são impressos diretamente no documento HTML sem sanitização com `htmlspecialchars()`, viabilizando ataques de XSS Refletido através de links forjados enviados aos usuários.

Evidências
- `public/index.php:104, 112`:
```php
echo $_GET['success_message'];
echo $_GET['error'];
```
- `admin/lista_livros.php:252, 261, 270`:
```html
value="<?php echo $filtroTitulo; ?>"
value="<?php echo $filtroAutor; ?>"
value="<?php echo $filtroAno; ?>"
```
- `public/cadastro.php:95`:
```html
<form method="POST" action="<?php echo $_SERVER['PHP_SELF']; ?>">
```

Impacto
Execução de scripts arbitrários no navegador da vítima, permitindo roubo de cookies de sessão (`PHPSESSID`), desfiguração de interface ou redirecionamentos maliciosos.

Sugestão de Correção
Aplicar `htmlspecialchars($valor, ENT_QUOTES, 'UTF-8')` em todas as variáveis refletidas no HTML:
```php
echo htmlspecialchars($_GET['success_message'], ENT_QUOTES, 'UTF-8');
```

```
#### Critérios de Aceite
- [ ] Nenhum parâmetro de URL é impresso sem escape no HTML.
- [ ] Payloads clássicos (ex: `<script>alert(1)</script>`) são neutralizados e renderizados como texto
puro.
--- FIM ISSUE 4 ---
```